Multi-Model AI 추론 운영 인프라 구축 및 최적화

요약

T4 GPU 한 장에서 LLM(Qwen2.5-1.5B, vLLM)과 CV(YOLO11n)를 함께 운영하면 두 워크로드 모두 성능이 저하됩니다. 단독 실행과 비교해 CV SLO 용량은 75 req/s 에서 25 req/s 로, LLM SLO 용량은 8 req/s 에서 4 req/s 로 감소했습니다. 하드웨어 변경 없이 설정 변경으로 LLM SLO 용량을 1.5 배(8 → 12 req/s), 경합 중 CV SLO 용량을 2 배(25 → 50 req/s)로 높였습니다.

- **병목**: 단독 실행에서는 LLM 은 vLLM 의 동시 처리 슬롯 32 개가, CV 는 워커 하나의 순차 처리가 처리량 상한을 결정합니다. 동시 실행에서는 프로세스 간 GPU 시분할이 주된 원인이며, 이미지 디코딩에 따른 CPU 경합이 더해집니다.
- **가설 검증**: 가설 5 개 중 3 개는 사전 예측대로 지지되었고 2 개는 기각되었습니다.
- **채택한 개선**: vLLM 슬롯 64 와 CV 워커 2 개입니다. 추가 비용 없이 같은 인스턴스에서 얻은 결과이며, Trade-off 는 TPOT 증가와 호스트 CPU 사용률 84%입니다.
- **장애 대응**: CV 워커 장애는 큐 적체 경보가 약 20 초 만에 탐지했습니다. 기본 구성(simple 큐)에서는 처리 중이던 작업 1 건이 유실되어, 재발 방지책으로 reliable 큐를 적용하고 같은 장애에서 유실 0 건을 확인했습니다. 재시작 후 60 초간 적체로 CV p95 가 22 초에 도달해, 워커가 없을 때 요청을 즉시 거절하는 방식(fail fast)을 다음 개선으로 제안합니다.
- **남은 과제**: 시분할을 피하는 구성(CUDA MPS, GPU 분리), CV 모델 크기에 따른 경합 변화 측정, 개선 사항을 통합한 구성의 재측정입니다.

1. 문제 정의와 범위

GPU 한 장(T4 16 GB)에서 LLM 과 CV 추론을 함께 운영할 때 어느 지점에서, 어떤 이유로 SLO 를 지키지 못하게 되는지 분석하고, 원인에 맞는 개선을 실험으로 검증하는 것이 목표입니다.

두 워크로드는 자원을 사용하는 방식이 다릅니다.

구분	LLM (Qwen2.5-1.5B-Instruct)	CV (YOLO11n 객체 탐지)
요청 하나의 처리 시간	수 초 (출력 128 토큰을 순차 생성)	수십 ms
사용자 체감 지표	첫 토큰까지의 시간(TTFT), 토큰 간 간격(TPOT)	전체 응답 시간
GPU 사용 방식	처리 중인 요청이 하나라도 있으면 GPU 를 계속 점유	요청이 있을 때만 짧은 배치를 실행
CPU 사용	적음	이미지 디코딩을 CPU 에서 수행
처리 구조	스트리밍 응답, vLLM 연속 배칭	비동기 큐 + 워커의 동적 배칭

따라서 핵심 질문은 "같은 GPU 를 공유하면 어느 워크로드가, 어떤 이유로, 얼마나 성능이 저하되는가"입니다.

자원	LLM 의 사용	CV 의 사용	경합 여부 (측정 근거)
GPU 연산	토큰 생성. 요청이 있으면 상시 점유	YOLO 추론	주된 경합. CV 이미지당 추론 5 → 18 ms, LLM TPOT +16~25% (5 장)
CPU	스케줄링·스트리밍 (적음)	이미지 디코딩, 큐 처리	부차적 경합. 경합 중 CV 디코딩 4.7 → 10.2 ms, 호스트 CPU 29 → 64% (5 장)
VRAM	가중치 + KV cache. 시작 시 50% 선점	YOLO 모델과 배치	경합 없음. 시작 시 고정 분할해 서로 침범하지 않음. KV cache 최대 8.3%, 전체 사용 8.4~9.1 / 16 GB
RAM	모델 로딩	이미지 버퍼	측정하지 않음 (9 장 한계)

분석 범위는 다음과 같습니다.

- 포함: 단일 호스트(Docker Compose)에서 두 워크로드 동시 운영, 부하 증가와 자원 경합 재현, 병목 분석, 설정 최적화, CV 워커 장애 재현과 복구
- 제외: 다중 노드 구성과 오토스케일링, 모델 정확도 개선, 생성 품질 평가 (부하 측정을 위해 출력 길이를 고정했습니다. 3 장 참고)

2. 측정 방법과 실험 환경

항목	내용
하드웨어	AWS EC2 g4dn.xlarge: NVIDIA T4 16 GB, vCPU 4, RAM 16 GB
부하 모델	포아송 도착
단계 구성	워밍업 10 초(집계 제외) → 측정 45 초 → 휴지 10 초. 전체 순서를 3 회 반복
LLM 요청	입력 약 64 단어, 출력 128 토큰 고정, 스트리밍
CV 요청	COCO val2017 이미지 업로드, 결과를 기다리는 동기 호출
처리량	측정 구간 안에 **완료된** 성공 요청 수 ÷ 45 초.
백분위수	요청별 기록에서 직접 계산. SLO 판정은 p95, 내부 동작 분석(배치·토큰 단위 비용)은 p50
자원 지표	측정 구간의 GPU·CPU·큐·vLLM 내부 상태를 Prometheus 에서 2 초 간격으로 수집해 저장
무효 처리	부하 생성기가 요청을 모두 전송하지 못했거나 예정보다 200 ms 이상 늦게 전송한 회차는 †로 표시하고 용량 판단에서 제외

출력 길이를 고정한 이유. LLM 처리 비용은 출력 길이에 비례합니다. 길이를 고정하면 요청당 작업량이 일정해 조건 간 비교가 공정해지고, 용량을 공식으로 예측할 수 있습니다(5 장).

무효 처리의 근거. 개방형 측정은 요청이 예정된 시각에 도착한다는 전제를 둡니다. 부하 생성기가 과부하로 전송이 늦어지면 실제 도착률이 표기값과 달라지고, 늦어진 시간만큼의 대기가 지연 기록에서 누락됩니다. 정상 단계의 전송 지연 p99 는 2~8 ms 이며, 기준값 200 ms 는 CV SLO(500 ms)의 40%입니다.

실험 목록

실험	과제 단계	내용
baseline	Baseline, 부하 증가, 경합 ①, 경합 ②	LLM 단독 2~12 req/s, CV 단독 25~100 req/s, LLM 4 req/s 고정 + CV 25~100 req/s CV 25 req/s 고정 + LLM 0.25~12 req/s
base-h2	H2-A 검증	경합 ② 측정과 동일 (CV 25 req/s 고정 + LLM 0.25~12 req/s)
e1-seqs64	H1 검증, 최적화	vLLM 슬롯 32 → 64, LLM 단독
e2a-batch16	H4 검증	CV 배치 8 → 16, 경합 ① 조건
e2b-workers2	H2-B 검증, 최적화	CV 워커 1 → 2, 경합 ① 조건
e5-simple, e5-reliable	장애 대응	부하 중 CV 워커 강제 종료 후 재시작. 재발 방지책(reliable)을 적용해 재검증

3. 운영 목표(SLO)

사용자 체감 지연은 p95 를 기준으로, 실패는 비율로 목표를 정했습니다.

지표	목표	근거
LLM TTFT p95	≤ 1,000 ms	사용자가 흐름이 끊겼다고 느끼기 시작하는 약 1 초
LLM TPOT p95	≤ 50 ms	초당 20 토큰 이상
CV 전체 지연 p95	≤ 500 ms	이미지를 올리고 결과를 기다리는 대화형 사용 기준
실패율	≤ 1%	5xx 응답과 연결 실패 포함

- **SLO 용량**: 모든 요청을 처리하면서(실패 1% 이하) 위 지연 목표를 만족한 가장 높은 도착률입니다. 최적화 전후는 이 값으로 비교합니다.
- 서비스 요구사항이 주어지지 않아 목표값은 직접 정했습니다. 목표값을 바꿔 분석 스크립트를 다시 실행할 수 있습니다(`--slo-ttft-ms` 등).

4. Baseline, Contention 측정

단독 실행에서는 LLM 은 vLLM 슬롯 32 개가, CV 는 워커 하나의 순차 처리(디코딩 → 추론)가 처리량 상한을 결정합니다. 두 워크로드를 동시에 실행하면 CV 는 GPU 실행 순서를 기다리게 되어 SLO 용량이 3 분의 1 로 감소하고, LLM 은 TPOT 이 증가해 SLO 용량이 절반으로 감소합니다.

정상 조건 정의

1. 경합 없음
2. 기본 설정, 장애 없음
3. 처리량 상한 이내의 부하

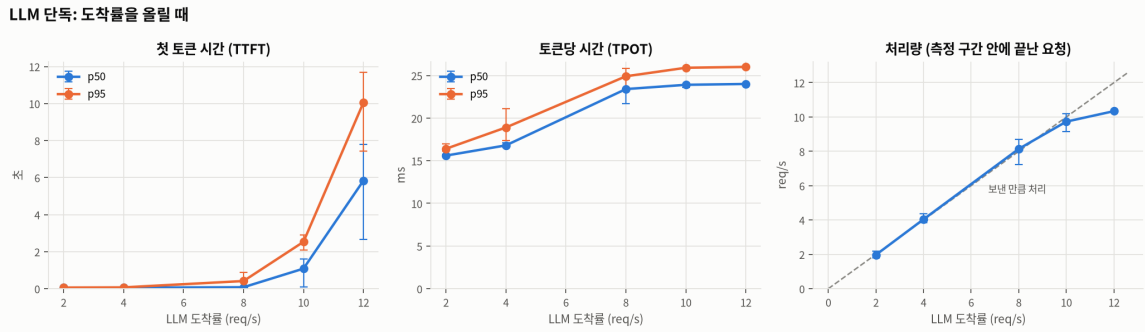
위 기준에 따라 LLM 4 req/s 와 CV 25 req/s 를 정상 조건으로 정하고 경합 실험의 고정값으로 사용했습니다. 실제 서비스에서는 운영 로그의 평균·피크 트래픽을 기준으로 정합니다.

Baseline 상태	LLM 4 req/s 단독	CV 25 req/s 단독
지연	TTFT p95 69 ms, TPOT p50 16.8 ms	p95 63 ms
GPU 사용률 (NVML)	100%	6%
GPU 메모리 (전체)	8.7 GB	8.7 GB
vLLM 처리 중 / 대기 / KV cache	최대 17 / 0 / 1.9%	해당 없음
CV 큐 길이 / 평균 배치	해당 없음	최대 2 / 1.72 장
호스트 CPU	31%	29%

GPU 메모리는 vLLM 이 시작 시 50%를 선점하므로 부하와 관계없이 일정합니다.
GPU 사용률은 LLM 요청이 하나만 처리 중이어도 100%로 표시되므로, 여유는 vLLM 대기 요청 수, KV cache 사용률, 지연으로 판단했습니다.

부하 증가

1. LLM 단독: vLLM 슬롯 수가 처리량 상한을 결정

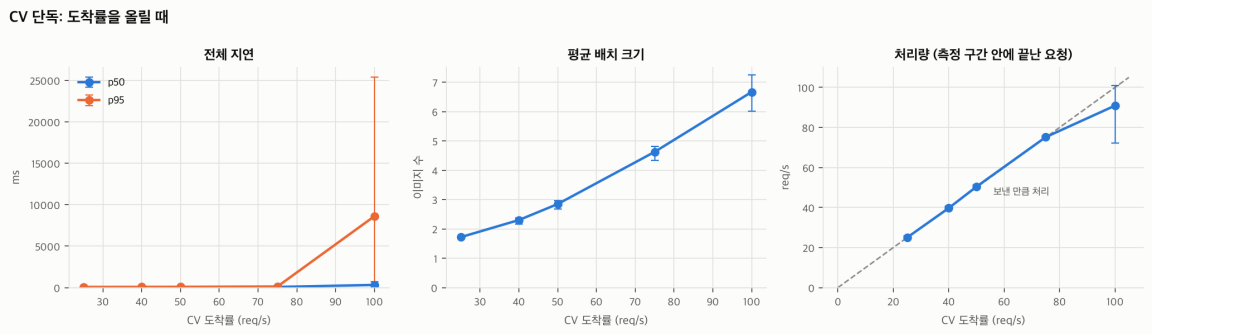


도착률 (req/s)	처리량 (req/s)	TTFT p50 (ms)	TTFT p95 (ms)	TPOT p50 (ms)	처리 중 최대	vLLM 대기 최대	KV cache 최대
2	1.97	52	60	15.6	8	0	0.9%
4	4.04	54	69	16.8	17	0	1.9%
8	8.14	77	412	23.4	31	4	3.6%
10	9.73	1,082	2,528	23.9	32	26	3.8%
12	10.34	5,836	10,052	24.0	32	32	3.8%

처리량은 8 req/s 까지 도착률과 같고, 12 req/s 에서 10.34 req/s(회차 범위 10.31~10.38)로 포화합니다. 이때 처리 중인 요청은 슬롯 수와 같은 32 개로 고정되고, KV cache 사용률은 최대 3.8%입니다. 따라서 처리량 상한은 메모리가 아니라 슬롯 수가 결정합니다. 도착률을 14·16 req/s 로 높여도 처리량은 약 10 req/s 를 넘지 않았습니다(7 장 E1 표). 리틀의 법칙으로 계산한 상한도 실측값과 일치합니다.

- 8 req/s 에서는 평균 약 24 개의 슬롯을 사용하지만, 요청이 순간적으로 몰리면 32 개를 넘습니다. 따라서 TTFT p95 가 회차에 따라 81~883 ms 로 달라지는 포화 직전 구간입니다.
- 10 req/s 부터 vLLM 내부에 대기 요청이 누적되어, TTFT p95 가 10 req/s 에서 2.5 초, 12 req/s 에서 10 초에 도달합니다.

2. CV 단독: 워커 하나의 순차 처리가 처리량 상한을 결정

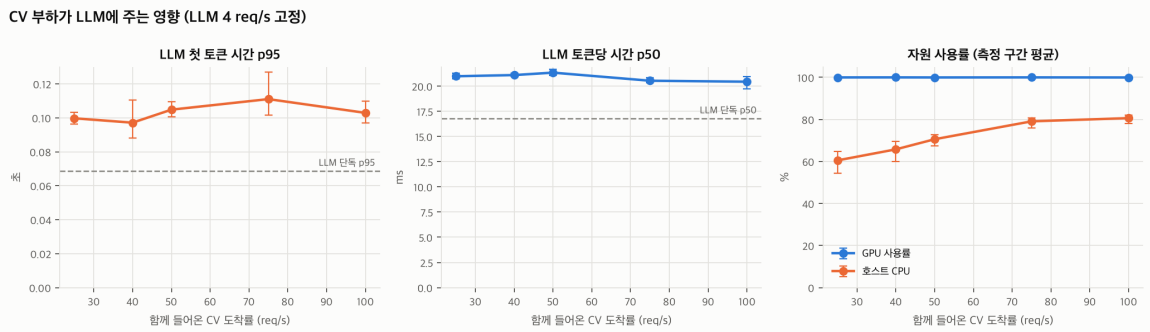


도착률 (req/s)	처리량 (req/s)	p95 (ms)	배치 추론 p50 (ms)	이미지당 추론 p50 (ms)	디코딩 p50 (ms)	평균 배치	큐 길이 최대	GPU 사용률
25	25.02	63	14.8	11.4	4.7	1.72	2	6%
40	39.71	75	16.7	7.9	6.5	2.30	3	9%
50	50.40	87	17.9	7.1	8.1	2.85	3	12%
75	75.17	128	23.0	5.3	14.2	4.63	5	17%
100 †	90.89	8,611	36.2	5.1	24.2	6.67	17	20%

- 75 req/s 까지는 p95 가 128 ms 이하로 여유가 있고, 100 req/s 는 회차에 따라 p95 가 0.2~25 초로 달라지는 포화 경계입니다.
- 도착률이 높아지면서 평균 배치가 1.7 장에서 6.7 장으로 커지고, 이미지당 추론 시간은 11.4 ms 에서 5.1 ms 로 감소했습니다. 단독 실행에서는 배칭으로 GPU 효율이 높아집니다.
- 디코딩은 이미지당 3.1~3.4 ms 로 일정하며, CPU 에서 한 장씩 순차로 수행됩니다. 100 req/s 에서는 배치 1 회 약 60 ms 중 24 ms(40%)가 디코딩이고, 이 시간 동안 GPU 는 유휴 상태입니다.
- GPU 사용률은 6~20%입니다. 단독 실행에서 CV 의 병목은 GPU 가 아니라 워커 구조입니다.

경합 재현

1. 경합 ①: LLM 4 req/s 고정 + CV 증가

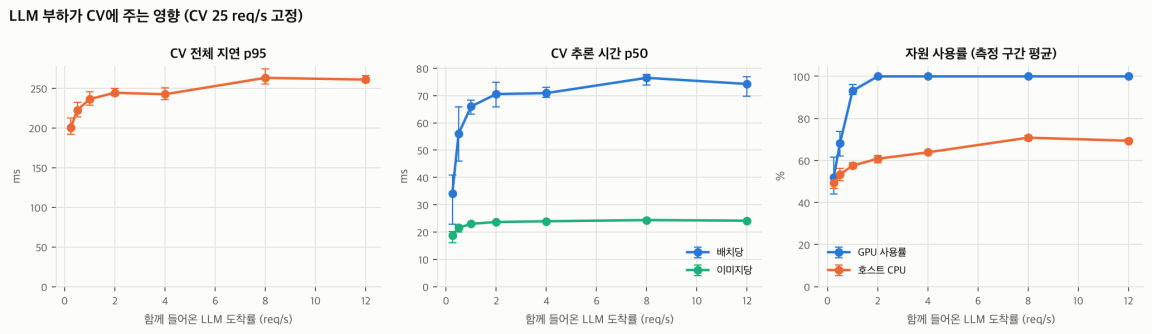


CV 도착률 (req/s)	CV 처리량	CV p95 (ms)	CV 배치 추론 p50 (ms)	CV 이미지당 추론 (ms)	CV 큐 최대	LLM TTFT p95 (ms)	LLM TPOT p50 (ms)
0 (LLM 단독)	-	-	-	-	-	69	16.8
25	24.96	240	69.4	23.7	3	100	21.0
40	39.84	513	136.6	18.9	21	97	21.1
50	44.50	7,656	141.8	17.7	357	105	21.3
75 †	32.37	41,642	106.8	17.0	448	111	20.5
100 †	31.76	48,273	56.3	16.5	430	103	20.4

- **CV 성능 저하.** CV 25 req/s 에서 배치 추론 시간이 단독 14.8 ms 에서 69.4 ms 로(4.7 배), p95 가 63 ms 에서 240 ms 로 증가했습니다. 배치가 커져도 이미지당 추론 시간은 17~19 ms 이하로 감소하지 않습니다(단독 약 5 ms). 처리량 상한은 약 44 req/s, SLO 용량은 25 req/s 입니다(40 req/s 에서 p95 513 ms, 회차 범위 441~633 ms).
- **LLM 성능 저하.** TTFT p95 는 69 ms 에서 97~111 ms 로, TPOT 은 16.8 ms 에서 20.4~21.3 ms 로(21~27%) 증가했지만 LLM SLO 는 만족했습니다.

- 75·100 req/s 의 처리량이 50 req/s 보다 낮은 이유는, 지연이 40 초를 넘어 측정 구간 안에 완료되지 못한 요청이 집계에서 제외되었기 때문입니다(무효 단계).

2. 경합 ②: CV 25 req/s 고정 + LLM 증가



LLM 도착률 (req/s)	LLM 처리량	LLM 출력 (tok/s)	LLM TTFT p95 (ms)	LLM TPOT p50 (ms)	CV p95 (ms)	CV 배치 추론 p50 (ms)	CV 이미지당 추론 (ms)
0 (CV 단독)	-	-	-	-	63	14.8	11.4
0.25	0.33	42	67	17.7	200	34.2	18.7
0.5	0.56	71	72	17.7	222	56.1	21.5
1	1.05	135	75	17.9	236	66.0	23.0
2	1.97	252	87	18.5	244	70.6	23.7
4	4.06	520	94	20.5	242	70.9	23.9
8	7.76	994	1,833	28.9	263	76.5	24.4
12	8.66	1,108	19,122	27.9	261	74.3	24.2

- **LLM 성능 저하.** CV 와 함께 실행하면 포화 시 TPOT 이 24.0 ms 에서 27.9 ms 로 증가하고, 처리량 상한은 10.3 req/s 에서 8.7 req/s 로 감소했습니다(공식: 32 ÷ (0.05 + 127 × 0.0279) ≈ 8.9 req/s). 8 req/s 에서 TTFT p95 가 1,833 ms 로 SLO 를 넘으므로, LLM SLO 용량은 8 req/s 에서 4 req/s 로 감소했습니다.
- 토큰당 증가분은 4 ms 이지만, 요청 하나가 토큰 127 개를 연속으로 생성하므로 슬롯 점유 시간은 0.5 초(16%) 증가하고 처리량 상한은 같은 비율로 감소합니다. 3 회 반복 범위(24.0 ms, 27.9~28.1 ms)가 겹치지 않으므로 측정 오차가 아닙니다. TPOT 28 ms 는 SLO(50 ms) 이내이므로, 이 증가는 사용자 체감보다 용량에 영향을 줍니다.
- **CV 성능 저하.** CV 배치 추론 시간은 LLM 2 req/s 까지 급격히 증가하고, 그 이후에는 거의 일정합니다. 원인이 LLM 작업량인지 GPU 점유 시간인지는 6 장(H2-A)에서 검증합니다.

5. 문제 발생 지점: 구간별 지연 분석

앞의 경합 ①·②는 두 워크로드의 성능이 얼마나 저하되었는지 측정한 결과입니다. 이 절에서는 같은 성능 저하가 요청 처리 경로의 어느 구간에서 발생했는지 나눠서 확인합니다. 원인이 무엇인지는 6 장에서 가설로 검증합니다.

CV. CV 지연 증가는 대부분 워커의 GPU 추론 단계에서 발생했고, CPU 디코딩 단계가 그다음입니다. CV 응답마다 기록되는 구간별 시간을 사용해, 같은 CV 25 req/s 를 단독 실행과 경합(LLM 4 req/s) 상황에서 비교했습니다.

구간	실행 자원	CV 단독	경합	변화
배치 추론 p50	GPU	14.8 ms	70.9 ms	4.8 배
이미지 디코딩 p50 (배치당)	CPU	4.7 ms	10.2 ms	2.2 배
큐 대기 p95	(앞 단계 지연의 결과)	20 ms	95 ms	4.8 배
전체 지연 p95	-	63 ms	242 ms	3.8 배
호스트 CPU	-	29%	64%	+35%p

- 추론 시간이 가장 크게 증가했습니다. 큐 대기 시간은 추론이 느려져 워커가 다음 배치를 늦게 가져가면서 함께 증가했습니다. 따라서 CV 의 첫 번째 문제 발생 지점은 GPU 입니다.
- 디코딩은 GPU 를 사용하지 않음에도 2.2 배 증가했습니다. vLLM 과 게이트웨이가 CPU 를 함께 사용하면서 생긴 CPU 경합이며, 두 번째 문제 발생 지점입니다(6 장 H2-B).

LLM. LLM 지연 증가도 GPU 공유에서 시작되지만, 드러나는 방식이 다릅니다. CV 와 GPU 를 함께 사용하면 토큰 하나를 생성하는 시간(TPOT)이 증가합니다. 이에 따라 요청 하나가 슬롯을 점유하는 시간이 증가하고, 새 요청이 빈 슬롯을 기다리는 시간도 증가합니다. 같은 LLM 도착률을 단독 실행과 경합(CV 25 req/s) 상황에서 비교했습니다.

구간	실행 자원	2 req/s: 단독 → 경합	8 req/s: 단독 → 경합
토큰 생성 (TPOT p50)	GPU	15.6 → 18.5 ms (+19%)	23.4 → 28.9 ms (+24%)
평균 사용 슬롯 (32 개 중, 추정)	vLLM 슬롯	4.1 → 4.8 개	24.2 → 29.8 개
첫 토큰 (TTFT p95)	슬롯 대기 + 프리필	60 → 87 ms	412 → 1,833 ms (4.4 배)

- 저부하(2 req/s)에서는 슬롯 여유가 커서 TTFT p95 증가가 27 ms 입니다.
- 8 req/s 에서는 평균 사용 슬롯이 32 개에 근접해, 요청이 순간적으로 몰리면 빈 슬롯을 기다리게 됩니다. TTFT p95 증가분(약 1.4 초)의 대부분은 이 대기 시간입니다. 따라서 LLM 의 문제 발생 지점은 GPU 공유로 길어진 슬롯 점유 시간입니다.
- 평균 사용 슬롯은 리틀의 법칙(도착률 × 요청당 처리 시간, 처리 시간 = 0.05 초 + 127 × TPOT)으로 계산한 추정값입니다. 경합 ② 측정에서는 vLLM 대기 요청 수를 집계하지 않았습니다.

6. 가설과 검증

본 실험에서는 원인으로 지목한 요소 하나만 바꿔 결과가 예측대로 변하는지 확인했습니다.

가설	검증 실험 (변경 요소)	사전 예측	결과	판정
H1. LLM 처리량 상한은 vLLM 슬롯 수(32)가 결정한다	E1: 슬롯 32 → 64	처리량 상한과 TPOT 이 함께 증가한다.	처리량 상한 10.34 → 13.88 req/s (공식 예측 13.96). SLO 용량 8 → 12. 포화 시 TPOT 24~26 → 35.7 ms. KV cache 최대 8.3%	지지
H2-A. CV 성능 저하의 원인은 LLM 의 GPU 점유 시간이다	base-h2: LLM 0.25~12 req/s	성능 저하가 LLM 점유율을 따르고, 점유율이 100%에 도달하는 약 2 req/s 이후에는 일정하다	점유율 예측 53 / 72 / 91 / 99%, 실측 GPU 사용률 52 / 68 / 93 / 100%. CV 배치 추론 34 → 56 → 66 → 71 ms 이후 71~77 ms. LLM 2 → 12 req/s 에서 LLM 출력은 4.4 배, CV 추론은 +5%	지지
H2-B. CPU 의 디코딩도 CV 처리량을 제한한다	E2-b: CV 워커 2 개	경합 중 CV 용량이 증가하고, 호스트 CPU 와 LLM TPOT 도 증가한다	SLO 용량 25 → 50 req/s. 50 req/s 에서 p95 7,656 → 299 ms. 호스트 CPU 71 → 84%, LLM TPOT 21.3 → 23.2 ms, VRAM +0.4 GB	지지
H3. 단일 파이썬 프로세스인 게이트웨이가 병목이다	기존 데이터: 게이트웨이 외부 소요 시간, 게이트웨이 CPU	병목이라면 부하에 따라 외부 소요 시간이 커지고 CPU 가 1 코어에 도달한다	CPU 최대 0.72 코어. 처리량 상한 이내에서 외부 소요 시간 p95 16 ms 이하	기각
H4. 경합 비용은 배치마다 일정하므로 큰 배치가 이를 분산한다	E2-a: 배치 16, 대기 30 ms	이미지당 추론 시간이 크게 감소하고 경합 중 용량이 증가한다	이미지당 추론 18.9 → 18.1 ms(−4%). 40 req/s p95 513 → 716 ms	기각

처리량 상한 ≈ 슬롯 수 ÷ (TTFT + 127 × TPOT)

7. 최적화 결과와 Trade-off

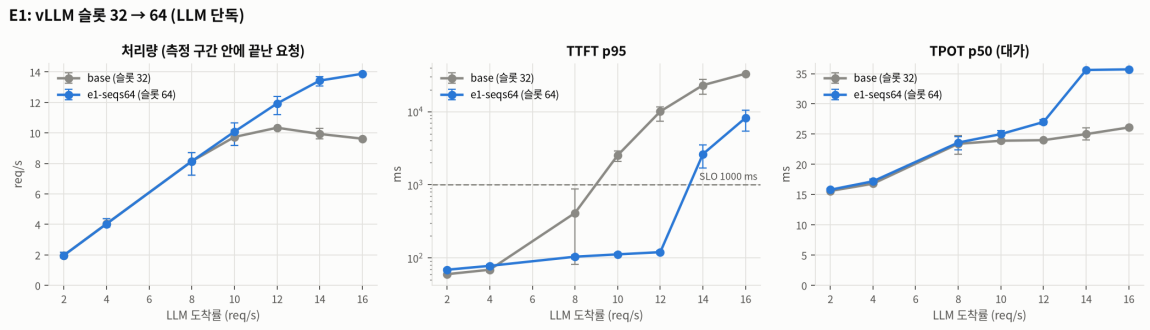
하드웨어 변경 없이 설정 변경으로 LLM SLO 용량을 1.5 배, 경합 중 CV SLO 용량을 2 배로 높였습니다.
모든 비교는 같은 세션의 `base` 결과를 기준으로 합니다.

설정	변경 내용	개선	Trade-off	판단
E1 슬롯 64	vLLM 슬롯 32 → 64, 게이트웨이 상한 64 → 128	LLM SLO 용량 8 → 12 req/s, 처리량 상한 10.3 → 13.9 req/s	TPOT 이 12 req/s 에서 24.0 → 27.0 ms, 포화 시 35.7 ms. 저부하 TTFT p95 +9 ms	채택. TPOT p95 31.6 ms 로 SLO 이내
E2-b 워커 2 개	CV 워커 1 → 2	경합 중 CV SLO 용량 25 → 50 req/s, 25 req/s p95 240 → 156 ms	호스트 CPU 71 → 84%, LLM TPOT +9%(50 req/s), VRAM +0.4 GB	채택. 다음 병목은 CPU
E2-a 배치 16	배치 8 → 16, 대기 10 → 30 ms	없음	p95 +27~40%	기각

E1: vLLM 설정(슬롯 32 → 64) 변경 실험 (LLM 단독)

도착률 (req/s)	처리량 (32 → 64)	TTFT p95 (ms)	TPOT p50 (ms)	처리 중 최대	KV cache 최대
8	8.14 → 8.14	412 → 104	23.4 → 23.6	31 → 36	3.6 → 4.6%
10	9.73 → 10.08	2,528 → 112	23.9 → 25.0	32 → 45	3.8 → 5.6%
12	10.34 → 11.93	10,052 → 120	24.0 → 27.0	32 → 57	3.8 → 7.0%
14	9.93 → 13.43	22,986 → 2,627	25.0 → 35.6	32 → 64	4.1 → 8.3%
16	9.62 → 13.88	32,996 → 8,222	26.1 → 35.7	32 → 64	4.3 → 8.3%

슬롯을 2 배로 늘렸지만 처리량 상한은 34% 증가했습니다.
한 스텝에서 처리하는 요청이 많아져 TPOT 이 24 ms 에서 35.7 ms 로 증가했기 때문이며,
다음 병목은 GPU 연산 자체입니다. KV cache 사용률은 최대 8.3%로 여전히 여유가 있습니다.



E2: 경합 중 CV 설정(배치, 워커 수) 변경 실험 (LLM 4 req/s 고정)

CV 전체 지연 p95 (ms):

CV 도착률 (req/s)	base (배치 8, 워커 1)	E2-a 배치 16	E2-b 워커 2
25	240	305	156
40	513	716	224
50	7,656	13,013 †	299
75	41,642 †	51,522 †	41,098 †
100	48,273 †	55,541 †	48,563 †
SLO 용량	25 req/s	25 req/s	50 req/s

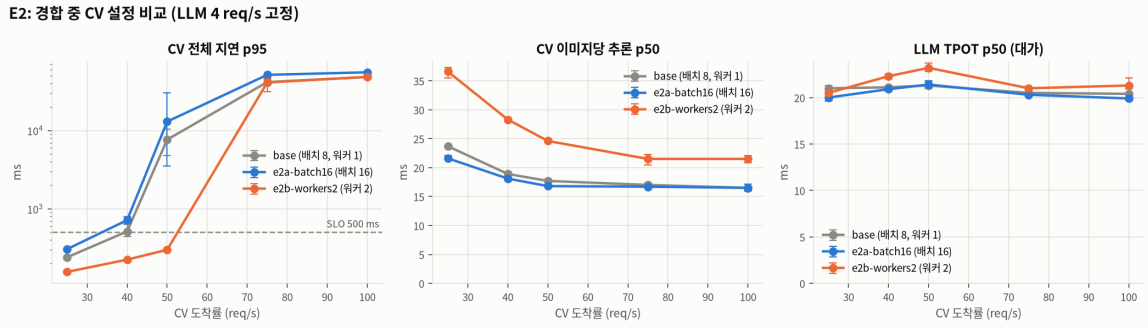
설정별 동작과 비용 (CV 40 req/s, 호스트 CPU·LLM TPOT 은 50 req/s):

항목	base	E2-a 배치 16	E2-b 워커 2
평균 배치 크기	6.79	12.14	2.82
이미지당 추론 시간 p50 (ms)	18.9	18.1	28.3
LLM TPOT p50 (ms)	21.3	21.4	23.2
호스트 CPU	71%	74%	84%
GPU 메모리 최대 (GB)	8.7	9.0	9.1

워커를 2 개로 늘리면 워커별 배치가 작아져 이미지당 추론 시간은 오히려 증가했습니다.

그러나 한 워커가 디코딩하는 동안 다른 워커가 추론할 수 있어 전체 처리량 상한이 증가했습니다.

배치 16 은 이미지당 추론 시간을 거의 줄이지 못하고 배치를 채우는 대기 시간만 증가시켰습니다(6 장 H4).



성능·비용·안정성 관점의 Trade-off

관점	E1 슬롯 64	E2-b 워커 2 개
성능	LLM SLO 용량 +50%, 처리량 상한 +34%	경합 중 CV SLO 용량 2 배, p95 −35~96% (25~50 req/s)
비용	추가 장비 없음. 대신 TPOT +13~37%(토큰 생성 속도 저하), KV cache 사용 증가	추가 장비 없음. 대신 호스트 CPU 84%, VRAM +0.4 GB

안정성	포화 지점이 약 10 → 14 req/s 로 이동해 부하 여유 증가. 포화 이후 TTFT 급증은 동일	SLO 위반 시작점이 40 → 50~75 req/s 로 이동. CPU 여유가 감소해 다음 병목이 될 위험

8. 장애 대응: CV 워커 장애

기본 구성(simple 큐)에서 부하 중 CV 워커가 중단되는 장애를 재현했습니다. 큐 적체 경보가 장애 발생 21 초 후 장애를 탐지했고, 재시작 후 41 초 만에 적체가 해소되었습니다. 처리 중이던 작업 1 건이 유실되어 재발 방지책으로 reliable 큐를 적용했고, 같은 장애를 다시 재현해 유실 0 건을 확인했습니다. 사용자에게 가장 큰 영향은 적체로 인한 지연(재시작 후 60 초간 p95 22 초)입니다.

① 재현

기본 구성(base, simple 큐)에서 CV 20 req/s 와 LLM 4 req/s 를 180 초 동안 전송하면서, 60 초에 CV 워커 컨테이너를 강제 종료하고 90 초에 재시작했습니다.

② 탐지

경보	조건	탐지 대상	발생 시각
CVQueueBacklog	큐 50 건 초과 및 지속 증가, 10 초	처리 중단의 결과	+21 초
CVWorkerDown	살아 있는 워커 0, 10 초	워커 생존 신호(heartbeat, TTL 15 초) 중단	+29 초
CVWorkerRestarted	2 분 내 워커 카운터 초기화	재시작 발생	+41 초
CVFailureRate	5xx 비율 1% 초과	사용자 대상 실패	+65 초
CVLatencySLO	CV p95 > 0.5 초, 30 초	SLO 위반	+73 초

③ 원인 분석

요청	결과	원인
장애 직후~25 초에 전송	200, 단 지연 20~40 초	큐에서 워커 재시작을 기다린 뒤 처리됨
장애 25~37 초에 전송	503 (187 건), 지연 정확히 5 초	결과 대기용 커넥션 풀(512 개)이 모두 사용됨
장애 순간 워커가 처리 중이던 작업	504 (1 건), 60 초 후	워커가 큐에서 꺼낸 직후 종료되어 작업이 유실됨.

세 번째 그룹이 작업 유실입니다. 기본 구성의 simple 큐는 워커가 작업을 꺼내는 순간 큐에서 삭제하므로, 처리 중 워커가 중단되면 해당 작업을 복구할 수 없습니다(at-most-once).

④ 복구

항목	값
종료부터 재시작 명령까지	31 초
장애 중 최대 큐 길이	717 건
적체 해소 (재시작 후, 큐 ≤ 5)	41 초
장애 전 CV p95	198 ms
재시작 후 60 초간 CV p95	22,052 ms

재시작된 워커는 모델을 로딩한 뒤 쌓인 작업을 최대 배치로 처리해 41 초 만에 적체를 해소했습니다.
게이트웨이가 이미 시간 초과로 응답한 작업은 워커가 건너뛰도록 구현해 해소 시간을 단축했습니다.

⑤ 재발 방지

대책	해결 대상	내용	상태
reliable 큐	작업 유실	꺼낸 작업을 "처리 중 목록"으로 옮기고 처리가 끝난 뒤 삭제합니다. 워커가 재시작할 때 남은 작업을 큐로 되돌려 처리합니다(at-least-once)	적용·검증
Fail fast	장애 중 사용자 대기	살아 있는 워커가 0 개이면 요청을 큐에 넣지 않고 즉시 503 을 반환합니다	제안
큐 길이 상한	적체 규모와 해소 시간	처리량 상한을 넘는 요청을 즉시 거절합니다	제안
탐지·복구 자동화	탐지·복구 시간	컨테이너 자동 재시작 정책과 더 짧은 heartbeat TTL 로 수동 재시작(31 초)과 생존 신호 탐지(29 초)를 단축합니다	제안

Reliable 큐 검증. 기본 구성에서 큐 모드만 reliable 로 바꿔 같은 장애를 재현했습니다

항목	simple	reliable
유실 (504)	1 건	0 건
재시작 시 복구한 작업	0 건	1 건

이번 조건(20 req/s, 평균 배치 1~3 장)에서는 차이가 1 건이지만, 배치 8 장을 처리하던 중 장애가 발생하면 최대 8 건, 워커가 여러 개면 그 배수까지 유실될 수 있습니다.

9. 한계와 개선 방향

결론은 단일 T4 와 합성 부하에서 얻은 결과이므로, 아래 조건이 달라지면 수치도 달라집니다.

한계

- 출력 128 토큰 고정: 실제 트래픽은 출력 길이가 다양해 꼬리 지연이 더 커질 수 있습니다.
- 경합 중 CV SLO 용량(25 req/s)은 측정한 도착률 간격에 따른 값입니다.
  실제 경계는 25 와 40 req/s 사이(40 에서 p95 513 ms)에 있습니다.
- 호스트 RAM 은 측정 지표에 포함하지 않았습니다.
  모델은 VRAM 에 상주해 RAM 경합 가능성은 낮지만, 측정으로 확인하지는 않았습니다.
- 부하 생성기가 같은 호스트에서 CPU 를 공유합니다.
  전송이 200 ms 이상 지연된 회차는 무효로 처리했습니다.
- GPU 사용률(NVML)은 커널이 하나라도 실행 중인 시간의 비율이므로, vLLM 이 동작하는 동안에는 항상 100% 근처입니다. 따라서 여유는 처리 중·대기 요청 수, KV cache 사용률, 지연으로 판단했습니다.
- 프로세스 간 GPU 전환의 정확한 비용은 드라이버 내부 동작이라 직접 측정하지 않았습니다.
- CV 모델 크기에 따른 경합 변화: 가장 작은 YOLO11n 만 측정했습니다. 모델이 커지면 CV 의 GPU 사용 시간이 증가해 CV 지연과 LLM TPOT 의 증가 폭이 모두 커질 것으로 예상합니다.

개선 방향

- CV 를 시분할에서 분리: CUDA MPS(여러 프로세스의 GPU 동시 사용),
- 워커 장애 시 fail fast(워커 0 이면 즉시 503), 자동 재시작과 짧은 heartbeat 주기
- 부하에 따라 슬롯·큐 상한을 조절하는 적응형 제어, 개선 사항을 통합한 구성의 재측정
- 실제 출력 길이 분포를 반영한 부하, 별도 호스트의 부하 생성기
- CV 모델 크기를 바꾼 경합 측정으로 모델 크기와 경합 정도의 관계 확인